

Interaction Classifier Architecture

The following Appendix provides a detailed description of the interaction classifier architecture. Figures 1 and 2 show the architectures for force and point cloud encoders, respectively. Further, Figure 3 shows the multimodal transformer architecture for fusing both modality representations, followed by the model classification head shown in Figure 4.

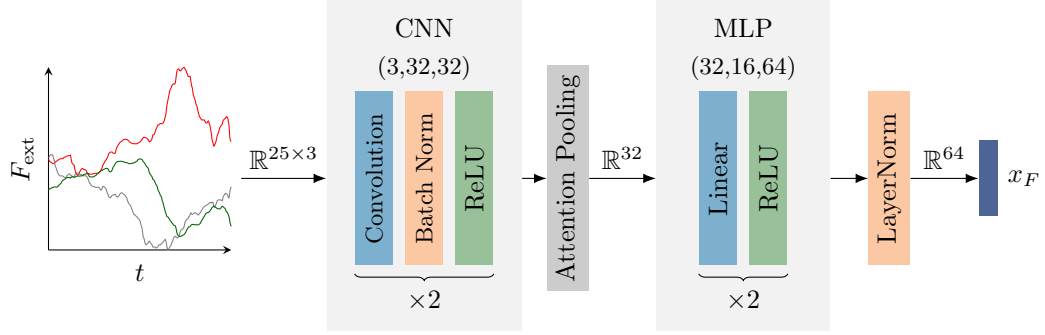


Figure 1: Force history encoder.

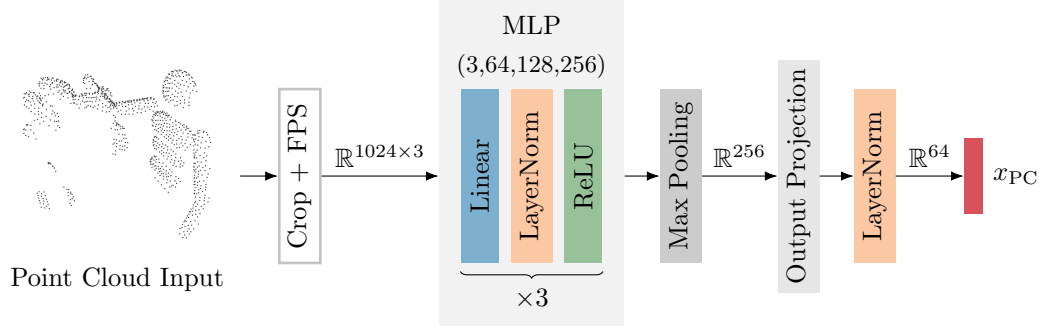


Figure 2: 3D Point Cloud Encoder [3].

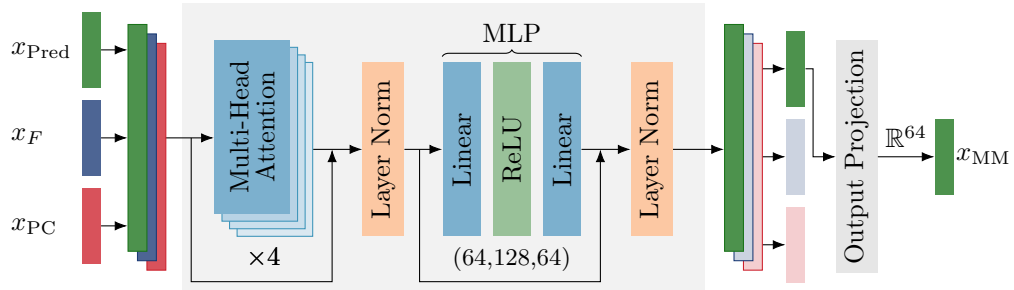


Figure 3: Multimodal Transformer.

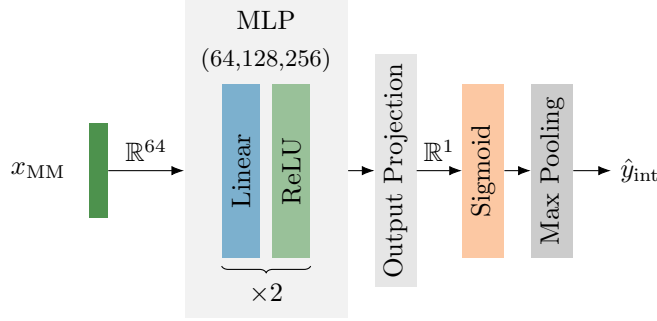


Figure 4: Classification Head.

Training Information and Experimental Setup

Tables 1 and 2 show training parameters for encoders and full model pretraining, respectively. Table 3 shows model fine-tuning training parameters. For all training stages, we utilized PyTorch Library [2] and workstation with an Intel[®] Core[™] Ultra 9 285K CPU and NVIDIA[®] GeForce RTX[™] 5090 GPU. To evaluate the model with real-time control loops on the Franka manipulator, we used a workstation with an Intel[®] Core[™] i7-11700K CPU and an NVIDIA[®] GeForce RTX[™] 3070 GPU, and RoBoStack [1] to establish software-controller-robot communications with ROS 1 Noetic.

	Learning Rate (η)	Epochs	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Time (min)
Force Encoder	0.0001	5000	0.162	0.201	1:53
Point Cloud Encoder	0.0001	1000	0.0455	0.0951	4:08

Table 1: Training parameters for encoder pretraining.

Encoders Learning Rate (η)	Transformer & Head Learning Rate (η)	Epochs	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Time (min)
0.00001	0.0001	1000	0.00737	0.0839	4:25

Table 2: Training parameters for model pretraining.

Dataset Composition (sim/real)	Learning Rate (η)	Epochs	$\mathcal{L}_{\text{train}}$	$\mathcal{L}_{\text{val}}$	Time (min)
10,000/25,000	0.00005	1000	0.0185	0.0479	11:57

Table 3: Training parameters for model fine-tuning.

Finally, we fine-tune the classifier with a mix of simulation-based and real data to facilitate sim-to-real transfer, while maintaining the generalization properties of simulation-based pretraining. However, since the bimanual setup is not within the training data, we decided to fine-tune the network for this experiment only on the real-robot dataset to better transfer to unseen morphologies. The classifier used in the bimanual experiment thus has the same architecture and pretraining, but instead of using a mix of simulated and real data for fine-tuning, it uses only the real data subset. We plan to create specialized model checkpoints tailored to specific morphologies in our future work, enabling more robust framework transfer.

References

- [1] Tobias Fischer, Wolf Vollprecht, Silvio Traversaro, Sean Yen, Carlos Herrero, and Michael Milford. A robostack tutorial: Using the robot operating system alongside the conda and jupyter data science ecosystems. *IEEE Robotics and Automation Magazine*, 2021.
- [2] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019.
- [3] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.